

Neural Network – Lab Report

Le Nghiem Thanh Thuy

1 Task of the source

The source build a simple feed-forward neural network from scratch, only use Python standard library (no NumPy, no PyTorch).

The program have to do two things:

- Read the network shape from a file `structure.txt` (number of layers, and number of neurons each layer).
- Two way to set the weights and bias:
 - Generate random value for weights and bias.
 - Or read them from file `parameters.txt`.
- After build the network, do a forward pass with a small input and print the output.

The activation function is sigmoid. The input layer do not have weights, it only pass the input to the next layer.

2 Source design

2.1 Class organization

The source is split into 3 class, each one in a separate file. The file `Main.py` is the entry point.

- **Neuron** (`Neuron.py`): the smallest unit. Each neuron hold a list of weights and one bias. It can initialize itself with random value, or set weights and bias from outside. The method `activate(inputs)` compute the weighted sum and apply sigmoid.
- **Layer** (`Layer.py`): a layer is a list of Neuron. When the layer do forward, it ask every neuron inside to activate and collect the output to a list.
- **NeuralNetwork** (`NeuralNetwork.py`): a list of Layer. This class read the structure file, create the layers, and provide method `forward`, `initialize_random_weights`, `load_parameters_from_file`, and `print_structure`.

The relation is simple: **NeuralNetwork** owns many **Layer**, and one **Layer** owns many **Neuron**.

2.2 Flow

The flow of the program is like this:

1. `Main.py` create a `NeuralNetwork` object.
2. Call `build_from_file("structure.txt")` – read number of layers and size of each layer, then build the `Layer` objects. The first layer is input layer, so its neurons have `input_size = 0`.
3. Set the parameters by one of two way:
 - `initialize_random_weights()` – random for every neuron except input layer.
 - `load_parameters_from_file("parameters.txt")` – read line by line, two line per neuron (one for weights, one for bias).
4. Call `forward(input_data)` – the input go through layer by layer (skip the input layer because it have no weights). Each layer return its output list, become the input of the next layer.
5. Print the final output.

The file `structure.txt` format: first line is the number of layers, then each next line is the size of one layer. The file `parameters.txt` format: for every neuron (except input layer), one line of weights separated by comma, and the next line is the bias.

3 Result

To test the network, we use it to solve the XOR problem. The network shape in `structure.txt` is set to 2-2-1 (2 input, 2 hidden, 1 output).

I choose weights and bias by hand so the hidden layer act like OR and NAND, and the output layer act like AND. Then the whole network compute XOR (because XOR = OR AND NAND). The parameters in `parameters.txt` is:

```
20, 20
-10
-20, -20
30
20, 20
-30
```

I test four input: (0,0), (0,1), (1,0), (1,1). The output is close to 0 for (0,0) and (1,1), and close to 1 for (0,1) and (1,0), which match the XOR truth table.

```

Building neural network from file...
-----

Loading parameters from file...
Neural Network Structure:
Layer 1: 2 neurons
  Neuron 1: Weights: [], Bias: 0.0
  Neuron 2: Weights: [], Bias: 0.0
Layer 2: 2 neurons
  Neuron 1: Weights: [20.0, 20.0], Bias: -10.0
  Neuron 2: Weights: [-20.0, -20.0], Bias: 30.0
Layer 3: 1 neurons
  Neuron 1: Weights: [20.0, 20.0], Bias: -30.0

Input 1: [0, 0]
(Debug) Layer forward outputs: [4.5397868702434395e-05, 0.9999999999999065]
(Debug) Layer forward outputs: [4.543910487654591e-05]
Output of the neural network: [4.543910487654591e-05]

Input 2: [0, 1]
(Debug) Layer forward outputs: [0.9999546021312976, 0.9999546021312976]
(Debug) Layer forward outputs: [0.999954519621495]
Output of the neural network: [0.999954519621495]

Input 3: [1, 0]
(Debug) Layer forward outputs: [0.9999546021312976, 0.9999546021312976]
(Debug) Layer forward outputs: [0.999954519621495]
Output of the neural network: [0.999954519621495]

Input 4: [1, 1]
(Debug) Layer forward outputs: [0.9999999999999065, 4.5397868702434395e-05]
(Debug) Layer forward outputs: [4.543910487654591e-05]
Output of the neural network: [4.543910487654591e-05]

```

Figure 1: Output of the network on four XOR input cases.